# Reflective Journal

Sam

Working through the Module 08 Bias-Variance Tradeoff Lab helped me visualize what actually happens under the hood when a model underfits or overfits data. By generating a synthetic sine wave with random noise and fitting Polynomial Regression models of degrees 1, 4, and 15, I could clearly see the difference between capturing a true pattern and memorizing noise. The Degree 1 model was a simple straight line that completely missed the curve of the sine wave, demonstrating high bias (underfitting). On the opposite extreme, the Degree 15 model contorted itself to hit individual noisy data points, showing high variance (overfitting). The Degree 4 polynomial hit the sweet spot, following the natural curve of the sine wave without overreacting to the noise.

Plotting the learning curves with scikit-learn's learning_curve function was the most valuable diagnostic takeaway for me. Seeing the Degree 1 training and cross-validation scores both flatten out at a very low score proved that feeding more training data into an underfit model will not fix its performance—the model architecture itself is too simple. Conversely, the massive gap between the near-perfect training score and the deeply negative cross-validation score on the Degree 15 learning curve showed exactly how an overfit model fails to generalize to unseen data. Along with the machine learning concepts, fixing the single-line syntax errors in the starter notebook reinforced my workflow for debugging code in Google Colab before committing the final notebook to GitHub.

AI Disclosure:

What AI was used: Google Gemini.

Why it was used: To help troubleshoot syntax errors caused by single-line code formatting in the starter notebook cells and to structure the pipeline code for polynomial regression and learning curves.

What I learned in reflection: I learned how polynomial degrees control model complexity, and how to read learning curves to diagnose whether a model is suffering from high bias (underfitting, where training and validation scores plateau low together) or high variance (overfitting, where a wide gap remains between high training and low validation scores).

How I modified it to make it my own: I formatted, executed, and verified all code blocks in my Google Colab environment, analyzed the resulting polynomial fit plots and 5-fold cross-validation learning curves, wrote out the knowledge check analysis, and committed the completed .ipynb file to my GitHub repository.

Ebuka

This lab helped me understand why a model that fits its training data well may still perform poorly on new data. The degree 1 model was too simple to capture the sine-wave pattern, while the degree 15 model had a large gap between its training and validation scores. I also learned that a model can look good in a plot and still be unreliable when tested on different samples. The main takeaway is to use validation results along with the plots when choosing model complexity, rather than assuming the most complex model is the best.